プログラミング３ 講義資料

第12回 チーム開発④：実装

- PR レビューの実践
- AI 活用とプロンプト記録
- 小さな PR を積み上げる

目次

- PR レビューの実践
- AI 活用プロンプト記録

PR レビューの実践

今回の目的と到達目標

今回の目的

- 実装を本格的に進める
- PR レビューを回す
- AI 記録を取りながら進める

到達目標

- [2] git/GitHub 活用
- [3] チーム協調

セルフレビューのチェックリスト

PR を出す前に自分で確認:

- [ ] 動作確認した（動かし方を PR に書いた）
- [ ] Issue 番号を紐付けた（`closes #N`）
- [ ] 不要な `console.log` やデバッグ文を消した
- [ ] Lint・フォーマットを通した
- [ ] 関係ない変更を混ぜていない
- [ ] コミットメッセージが「何を/なぜ」になっている

レビュワーのチェックリスト

- [ ] PR の目的が明確か
- [ ] コードが読みやすいか（命名・構造）
- [ ] Issue の受け入れ条件を満たすか
- [ ] 影響範囲が想定内か
- [ ] テスト/動作確認の根拠が書いてあるか
- [ ] セキュリティ上の問題はないか（機密の混入 等）

> 「良さそう」で Approve しない — 根拠を書く

コメントの書き方

プレフィックス	意味
`must:`	必ず直してほしい
`should:`	直した方がよい
`nit:`	些細な指摘（直さなくても可）
`question:`	質問・確認
`praise:`	良かった点

- 温度感を揃えると議論がスムーズ

AI 活用プロンプト記録

記録すべきこと

すべての提出物で、AI を使った場合は以下を記録:

項目	内容
日時	いつ投げたか
プロンプト	実際に送った文
AI の出力	返ってきた内容
採否	採用 / 棄却 / 加筆
理由	なぜその判断か

記録フォーマット例

```
## 2026-06-15 投稿一覧画面のコード生成

**プロンプト**:
Express + EJSで投稿一覧を表示する。
posts テーブルから全件取得して render する関数を書いて。

**出力（要約）**:
- `app.get('/posts', ...)` のルーティング
- `db.all('SELECT * FROM posts', ...)` で取得
- 非同期コールバック版

**採否**: 加筆採用
**理由**: async/await に書き直し、エラーハンドリングを追加
```

AI を使う際の注意

Tips

- **そのまま貼らない** — 内容を読んで判断する
- **秘密情報を投げない** — 認証情報・個人情報・未公開仕様
- **ライセンス** — 出力コードの扱いを確認
- 採用する場合でも、自分で理解できていないコードは入れない
- レビューでは「どこを AI に頼ったか」を共有する

今回のまとめ

- セルフレビュー → 依頼 → レビュー → 修正 → merge のサイクル
- コメントは `must:` / `should:` / `nit:` で温度感を揃える
- AI 活用は **プロンプト・出力・採否・理由** を記録する

今回カバーした MCC 項目

- V-D-1 プログラミング
- V-D-4 コンピュータシステム

次回予告

- 第13回: チーム開発⑤ 実装・テスト・AI活用記録共有